

AI-Enabled Capstone Project Requirements Checklist

ICT Training Institution - Post Graduate Full Stack Development Tracks

Version 1.0 | January 2026

PROJECT OVERVIEW REQUIREMENTS

☒ Core Project Criteria

- ☐ **Real-world Problem:** Solves an actual business, educational, social, or technical problem
- ☐ **MVP Quality:** Production-ready minimum viable product, NOT a demo or proof of concept
- ☐ **Market Readiness:** Feature-complete enough to be marketed or serve as startup foundation
- ☐ **Team Collaboration:** Demonstrates coordinated work across 5 team members
- ☐ **4-Week Timeline:** Deliverable within development and testing period
- ☐ **Bilingual Support:** Supports English and/or Arabic (based on target audience)

☒ Technical Stack Foundation

- ☐ **Full Stack Architecture:** Complete frontend, backend, and database implementation
 - ☐ **Version Control:** Comprehensive Git repository with meaningful commits
 - ☐ **CI/CD Pipeline:** Automated build, test, and deployment process
 - ☐ **Cloud Deployment:** Hosted on production infrastructure (AWS/Azure/GCP)
 - ☐ **Documentation:** Complete technical documentation, API docs, and user guides
-

MANDATORY AI/GEN-AI FEATURES

1. LLM Integration & APIs (MUST-HAVE)

A. Foundation Model Integration

- ☐ **Primary LLM Provider:** OpenAI, Google Gemini, Anthropic Claude, or Azure OpenAI
- ☐ **Model Selection Justification:** Document why specific model(s) chosen
- ☐ **Fallback Strategy:** Secondary model or graceful degradation plan
- ☐ **API Key Management:** Secure credential storage (environment variables, secrets manager)

B. Core LLM Capabilities

- ☐ **Text Generation:** Completions, chat, or conversational interface

- ☐ **Structured Output:** JSON mode or structured data extraction
- ☐ **System Prompts:** Well-designed system instructions with role definition
- ☐ **Context Management:** Proper handling of conversation history
- ☐ **Token Management:** Track and optimize token usage

C. Advanced API Features (Choose minimum 2)

- ☐ **Function Calling:** Tool use with external APIs or databases
 - ☐ **Streaming Responses:** Real-time output generation
 - ☐ **Batch Processing:** Handling multiple requests efficiently
 - ☐ **Embeddings API:** Text embeddings for semantic tasks
 - ☐ **Fine-tuned Models:** Custom model fine-tuning for domain specificity
-

2. RAG (Retrieval-Augmented Generation) IMPLEMENTATION (MUST-HAVE)

A. Document Processing Pipeline

- ☐ **Multi-format Support:** PDFs, DOCX, TXT, HTML, markdown
- ☐ **Document Loader:** Automated ingestion from multiple sources
- ☐ **Text Extraction:** OCR support for scanned documents and images
- ☐ **Metadata Extraction:** Document properties, dates, authors, categories

B. Chunking Strategy

- ☐ **Intelligent Chunking:** Semantic, hierarchical, or sliding window
- ☐ **Chunk Size Optimization:** Tested optimal size (typically 512-1024 tokens)
- ☐ **Overlap Strategy:** 10-20% overlap between chunks
- ☐ **Preserve Context:** Maintain headers, references, and structural information

C. Vector Database & Embeddings

- ☐ **Vector Store Selection:** Pinecone, Weaviate, Chroma, FAISS, Qdrant, or pgvector
- ☐ **Embedding Model:** OpenAI ada-002, sentence-transformers, or domain-specific
- ☐ **Dimensionality:** Appropriate vector dimensions (768-1536)
- ☐ **Indexing Strategy:** Efficient similarity search (HNSW, IVF)
- ☐ **Metadata Filtering:** Filter results by date, category, source, etc.

D. Retrieval Strategy (Implement minimum 2)

- ☐ **Semantic Search:** Dense vector similarity
- ☐ **Keyword Search:** BM25 or traditional lexical search

- ☐ **Hybrid Retrieval:** Combine semantic + keyword with score fusion
- ☐ **Reranking:** Cross-encoder or LLM-based reranking
- ☐ **MMR (Maximal Marginal Relevance):** Diversity in retrieved results
- ☐ **Parent-Child Retrieval:** Retrieve chunks but provide larger context

E. RAG Quality & Evaluation

- ☐ **Context Precision:** Measure relevance of retrieved chunks
- ☐ **Context Recall:** Measure completeness of retrieval
- ☐ **Faithfulness:** Answers grounded in retrieved context
- ☐ **Answer Relevancy:** Output directly addresses query
- ☐ **Citation/Source Attribution:** Reference source documents in responses

F. Advanced RAG Features (Choose minimum 1)

- ☐ **Query Transformation:** Query expansion, decomposition, or rephrasing
 - ☐ **Multi-query RAG:** Generate multiple queries from single input
 - ☐ **Hypothetical Document Embeddings (HyDE):** Generate hypothetical answers for retrieval
 - ☐ **Corrective RAG:** Verify and refine retrieved context
 - ☐ **Self-RAG:** Model evaluates own retrieval and generation
 - ☐ **Adaptive RAG:** Route to different retrieval strategies based on query
-

3. AGENTIC AI IMPLEMENTATION (MUST-HAVE)

A. Single Agent Foundation

- ☐ **Agent Framework:** LangChain Agents, AutoGen, Semantic Kernel, CrewAI, or similar
- ☐ **Agent Architecture:** ReAct, Plan-and-Execute, or Function Calling
- ☐ **Goal Definition:** Clear agent objectives and success criteria
- ☐ **Reasoning Capability:** Chain-of-thought or step-by-step reasoning

B. Agent Tool Use & Function Calling

- ☐ **Tool/Function Registry:** Defined set of available tools
- ☐ **Minimum 3 Tools:** Database queries, web search, calculations, API calls, etc.
- ☐ **Tool Descriptions:** Clear documentation for LLM to understand tool usage
- ☐ **Error Handling:** Graceful failures when tools fail
- ☐ **Tool Execution Tracking:** Log which tools are called and why

C. Agent Memory & Context

- ☐ **Short-term Memory:** Conversation/session context
- ☐ **Long-term Memory:** Persistent storage across sessions (vector memory or database)
- ☐ **Episodic Memory:** Store and retrieve past interactions
- ☐ **Context Window Management:** Handle long conversations with summarization

D. Multi-Agent Systems (Implement at least ONE approach)

Option 1: Sequential Agents (Pipeline)

- ☐ **Agent Workflow:** 3+ agents working in sequence
- ☐ **Task Handoff:** Clear output from one agent becomes input to next
- ☐ **Supervisor Agent:** Orchestrates workflow between agents
- ☐ **Example:** Research → Analysis → Report Generation

Option 2: Parallel Agents (Collaborative)

- ☐ **Concurrent Execution:** Multiple agents work simultaneously
- ☐ **Result Aggregation:** Combine outputs from different agents
- ☐ **Example:** Multiple agents analyzing different data sources

Option 3: Hierarchical Agents (Manager-Worker)

- ☐ **Manager Agent:** Breaks down complex tasks
- ☐ **Worker Agents:** Specialized agents for subtasks (minimum 2 workers)
- ☐ **Dynamic Delegation:** Manager assigns tasks based on agent capabilities

Option 4: Swarm/Consensus Agents

- ☐ **Multiple Perspectives:** Same task handled by multiple agents
- ☐ **Voting/Consensus:** Aggregate agent outputs for final decision
- ☐ **Quality Improvement:** Better results through diversity

E. Agent Evaluation & Monitoring

- ☐ **Task Success Rate:** Percentage of successfully completed tasks
- ☐ **Tool Usage Analytics:** Which tools are most/least used
- ☐ **Reasoning Trace:** Log agent's decision-making process
- ☐ **Latency Tracking:** Time per agent action and overall task
- ☐ **Cost Attribution:** Token usage per agent and tool call

F. Advanced Agent Features (Choose minimum 2)

- ☐ **Human-in-the-Loop:** User approval for critical actions
- ☐ **Agent Self-Reflection:** Agent evaluates own output quality

- ☐ **Dynamic Replanning:** Adjust strategy when initial approach fails
 - ☐ **Multi-modal Agents:** Process text, images, audio, or video
 - ☐ **Agent Learning:** Improve from user feedback or past interactions
-

4. MULTIMODAL AI CAPABILITIES (Choose minimum 2)

A. Vision/Image Understanding

- ☐ **Image Analysis:** GPT-4V, Gemini Vision, Claude 3 Vision, or LLaVA
- ☐ **Use Cases:** Document analysis, image Q&A, visual search, accessibility
- ☐ **Image Upload:** Support for JPG, PNG, WebP
- ☐ **Batch Processing:** Handle multiple images

B. Image Generation

- ☐ **Model Selection:** DALL-E 3, Stable Diffusion, Midjourney API, or Imagen
- ☐ **Prompt Engineering:** Optimized prompts for quality generation
- ☐ **Style Controls:** Allow user customization
- ☐ **Safety Filters:** Content moderation for generated images

C. Speech-to-Text (STT)

- ☐ **Transcription Service:** Whisper, Google Speech-to-Text, or Azure Speech
- ☐ **Multi-language Support:** At least English and Arabic
- ☐ **Real-time vs Batch:** Choose based on use case
- ☐ **Speaker Diarization:** Identify different speakers (if applicable)

D. Text-to-Speech (TTS)

- ☐ **Voice Synthesis:** OpenAI TTS, ElevenLabs, Azure Speech, or Google TTS
- ☐ **Multiple Voices:** Offer voice selection
- ☐ **Emotion/Tone:** Control speaking style
- ☐ **Streaming Audio:** Real-time playback

E. Video Understanding

- ☐ **Video Processing:** Gemini 1.5 Pro, GPT-4V with frames, or Claude 3
- ☐ **Frame Extraction:** Sample key frames for analysis
- ☐ **Temporal Understanding:** Understand sequence of events
- ☐ **Use Cases:** Video summarization, content moderation, scene detection

F. Audio Analysis

- ☐ **Audio Classification:** Emotion detection, sound recognition
 - ☐ **Music Understanding:** Genre, mood, instrument detection
 - ☐ **Audio Events:** Environmental sound detection
-

🔧 5. LLM ORCHESTRATION & FRAMEWORKS (MUST-HAVE)

A. Framework Selection (Choose ONE primary framework)

- ☐ **LangChain:** Python/TypeScript, comprehensive ecosystem
- ☐ **LangGraph:** Stateful multi-agent workflows
- ☐ **LlamaIndex:** Data-focused, excellent for RAG
- ☐ **Semantic Kernel:** Microsoft C#/.NET focus
- ☐ **AutoGen:** Microsoft multi-agent framework
- ☐ **CrewAI:** Role-based multi-agent systems
- ☐ **DSPy:** Programmatic prompting and optimization
- ☐ **Haystack:** Production-ready pipelines

B. Framework Implementation

- ☐ **Modular Architecture:** Separate retrieval, generation, orchestration
- ☐ **Chain Management:** Well-structured prompting chains
- ☐ **Error Boundaries:** Graceful failure handling at each step
- ☐ **Logging Integration:** Comprehensive trace logging
- ☐ **Testing Suite:** Unit tests for chains and agents

C. Prompt Management

- ☐ **Centralized Prompts:** Store prompts in database or config files
 - ☐ **Version Control:** Track prompt changes over time
 - ☐ **A/B Testing:** Compare prompt variations
 - ☐ **Prompt Templates:** Parameterized prompts with variables
 - ☐ **Few-shot Examples:** Include demonstration examples in prompts
-

🔒 6. SECURITY, SAFETY & COMPLIANCE (MUST-HAVE)

A. OWASP LLM Top 10 2025 Coverage

1. Prompt Injection Defense

- ☐ **Input Sanitization:** Filter malicious instructions
- ☐ **System Prompt Protection:** Prevent system prompt extraction
- ☐ **Direct Injection Defense:** Block attempts to override instructions
- ☐ **Indirect Injection Defense:** Protect against poisoned documents/data

2. Sensitive Information Disclosure Prevention

- ☐ **Data Sanitization:** Remove PII from training/retrieval data
- ☐ **Output Filtering:** Scan responses for sensitive information
- ☐ **User Opt-out:** Privacy controls and data deletion
- ☐ **Audit Logging:** Track what data is accessed and exposed

3. Supply Chain Security

- ☐ **Dependency Scanning:** Regularly audit third-party packages
- ☐ **Model Provenance:** Verify source of models and datasets
- ☐ **Version Pinning:** Lock dependency versions
- ☐ **Vulnerability Monitoring:** Track CVEs in dependencies

4. Excessive Agency Controls

- ☐ **Action Limits:** Rate limiting on agent actions
- ☐ **Approval Workflows:** Human-in-the-loop for critical operations
- ☐ **Scope Restrictions:** Limit what agents can access/modify
- ☐ **Rollback Mechanisms:** Undo agent actions if needed

5. Insecure Output Handling

- ☐ **Output Validation:** Check generated code before execution
- ☐ **Sandbox Execution:** Run untrusted code in isolated environment
- ☐ **XSS Prevention:** Sanitize outputs displayed in web UI
- ☐ **SQL Injection Prevention:** Parameterized queries only

6-10. Additional Protections

- ☐ **Data Poisoning Defense:** Validate training/retrieval data quality
- ☐ **Model DoS Protection:** Rate limiting, request size limits
- ☐ **Model Theft Prevention:** API key rotation, usage monitoring
- ☐ **Vector DB Security:** Encrypted embeddings, access controls

B. Guardrails & Content Moderation

- ☐ **Input Guardrails:** Block harmful, inappropriate, or off-topic inputs
- ☐ **Output Guardrails:** Filter harmful, biased, or incorrect outputs
- ☐ **Content Moderation:** OpenAI Moderation API, Azure Content Safety, or custom
- ☐ **Toxicity Detection:** Flag toxic, hateful, or discriminatory content
- ☐ **PII Detection:** Identify and redact personal information
- ☐ **Factuality Checking:** Verify claims against trusted sources (when applicable)

C. Rate Limiting & Abuse Prevention

- ☐ **Per-user Rate Limits:** Prevent individual abuse
- ☐ **Global Rate Limits:** Protect backend infrastructure
- ☐ **Token Budgets:** Limit per-request and per-user token usage
- ☐ **IP-based Throttling:** Detect and block suspicious traffic
- ☐ **CAPTCHA Integration:** Prevent bot abuse

D. Data Privacy & Compliance

- ☐ **Data Encryption:** Encrypt data at rest and in transit (TLS/SSL)
 - ☐ **User Consent:** Clear terms for data usage
 - ☐ **Right to Deletion:** Users can delete their data
 - ☐ **Data Retention Policy:** Define how long data is stored
 - ☐ **Anonymization:** Remove identifying information where possible
 - ☐ **GDPR/CCPA Readiness:** Basic compliance with privacy regulations
-

7. MONITORING, OBSERVABILITY & EVALUATION (MUST-HAVE)

A. LLM Observability Platform (Choose ONE)

- ☐ **Langfuse:** Open-source, comprehensive tracing
- ☐ **LangSmith:** LangChain-native, production-grade
- ☐ **Arize AI:** Enterprise observability and monitoring
- ☐ **Braintrust:** Evaluation-first approach
- ☐ **Weights & Biases (Weave):** LLM tracking and evaluation
- ☐ **LangWatch:** Agent-specific observability
- ☐ **Helicone:** Simple observability and caching
- ☐ **Phoenix (Arize OSS):** Open-source alternative

B. Trace & Logging

- ☐ **Distributed Tracing:** Full request path visibility (OpenTelemetry compatible)

- ☐ **Span Instrumentation:** Capture retrieval, LLM calls, tool use, agent actions
- ☐ **Input/Output Logging:** Store prompts and completions
- ☐ **Metadata Capture:** User ID, session ID, model, temperature, tokens
- ☐ **Error Tracking:** Log failures with stack traces
- ☐ **Search & Filter:** Query logs by user, time, model, error type

C. Performance Metrics

- ☐ **Latency Tracking:** Response time per component (retrieval, LLM, total)
- ☐ **Token Usage:** Input tokens, output tokens, cached tokens, reasoning tokens
- ☐ **Cost Attribution:** Calculate cost per request, per user, per model
- ☐ **Throughput:** Requests per second, concurrent users
- ☐ **Error Rates:** Track failures by type and component
- ☐ **Time-to-First-Token (TTFT):** For streaming responses

D. Quality Evaluation (Implement minimum 3 metrics)

Retrieval Quality

- ☐ **Context Precision:** Relevance of retrieved chunks
- ☐ **Context Recall:** Completeness of retrieval
- ☐ **MRR (Mean Reciprocal Rank):** Rank of first relevant result

Generation Quality

- ☐ **Faithfulness/Groundedness:** Answers based on retrieved context
- ☐ **Answer Relevancy:** Response addresses the query
- ☐ **Coherence:** Logical flow and readability
- ☐ **Conciseness:** Appropriate length and focus

LLM-as-a-Judge

- ☐ **Automated Scoring:** Use GPT-4/Claude to score outputs
- ☐ **Custom Rubrics:** Define evaluation criteria
- ☐ **Pairwise Comparison:** Compare multiple outputs
- ☐ **Reference-based Eval:** Compare against golden answers

Human Evaluation

- ☐ **Thumbs Up/Down:** Simple user feedback
- ☐ **Star Ratings:** 1-5 scale for quality
- ☐ **Detailed Feedback:** Text comments from users
- ☐ **Human Labeling Queue:** Team reviews flagged responses

E. Agent-Specific Monitoring

- ☐ **Task Success Rate:** Percentage of completed tasks
- ☐ **Tool Usage Analytics:** Most/least used tools, success rates
- ☐ **Reasoning Trace Visibility:** Show agent's thought process
- ☐ **Multi-agent Coordination:** Track handoffs and communication
- ☐ **Agent Cost Attribution:** Tokens/cost per agent type

F. Alerting & Anomaly Detection

- ☐ **Quality Alerts:** Notify when eval metrics drop below threshold
- ☐ **Cost Alerts:** Warn when spending exceeds budget
- ☐ **Error Alerts:** Immediate notification for critical failures
- ☐ **Latency Alerts:** Flag slow responses
- ☐ **Usage Alerts:** Detect unusual traffic patterns

G. Continuous Improvement Loop

- ☐ **Production → Evaluation:** Failed cases become test datasets
 - ☐ **Evaluation → Development:** Insights drive prompt/model improvements
 - ☐ **A/B Testing:** Compare prompt/model variations in production
 - ☐ **Regression Testing:** Prevent quality degradation on updates
 - ☐ **Feedback Integration:** Use user feedback to improve system
-

8. USER EXPERIENCE & INTERFACE (MUST-HAVE)

A. Conversational Interface

- ☐ **Chat UI:** Clean, intuitive messaging interface
- ☐ **Markdown Support:** Render formatted text, code blocks, tables
- ☐ **Streaming Responses:** Real-time token-by-token display
- ☐ **Typing Indicators:** Show when AI is "thinking"
- ☐ **Message History:** Persist and display conversation
- ☐ **Copy/Share Features:** Easy sharing of AI responses

B. File Upload & Management

- ☐ **Drag-and-drop Upload:** Intuitive file selection
- ☐ **Multi-file Support:** Upload multiple documents at once
- ☐ **Progress Indicators:** Show upload and processing status

- ☐ **File Previews:** Display thumbnails or text previews
- ☐ **File Management:** View, delete, re-process uploaded files
- ☐ **Supported Formats:** Clear indication of accepted file types

C. User Feedback Mechanisms

- ☐ **Inline Feedback:** Thumbs up/down on each response
- ☐ **Report Issues:** Flag inappropriate or incorrect outputs
- ☐ **Explain More:** Request elaboration or clarification
- ☐ **Regenerate:** Get alternative responses
- ☐ **Follow-up Suggestions:** Suggest relevant next questions

D. Advanced UI Features (Choose minimum 2)

- ☐ **Citation Bubbles:** Inline source references with click-to-view
- ☐ **Source Documents Panel:** Side panel showing retrieved context
- ☐ **Confidence Scores:** Display model's confidence in answer
- ☐ **Related Questions:** Suggest relevant follow-up queries
- ☐ **Export Chat:** Download conversation as PDF, TXT, or markdown
- ☐ **Voice Interface:** Speech input and audio output
- ☐ **Multi-language Toggle:** Switch between English and Arabic

E. Responsive Design

- ☐ **Mobile-First:** Optimized for smartphones
- ☐ **Tablet Support:** Adapted layout for tablets
- ☐ **Desktop Experience:** Full-featured desktop interface
- ☐ **Progressive Web App (PWA):** Installable, offline-capable (if applicable)

F. Accessibility (WCAG 2.1 Level AA)

- ☐ **Screen Reader Support:** Semantic HTML, ARIA labels
 - ☐ **Keyboard Navigation:** Full functionality without mouse
 - ☐ **Color Contrast:** Sufficient contrast ratios
 - ☐ **Text Scaling:** Supports browser zoom up to 200%
 - ☐ **Alt Text:** Descriptive text for images and icons
 - ☐ **Focus Indicators:** Visible focus states
-

9. COST OPTIMIZATION & SCALABILITY (MUST-HAVE)

A. Cost Management

- ☐ **Token Optimization:** Minimize unnecessary tokens in prompts
- ☐ **Caching Strategy:** Cache embeddings, frequent queries, or intermediate results
- ☐ **Model Selection:** Use appropriate model for task (avoid GPT-4 when 3.5 suffices)
- ☐ **Batch Processing:** Group requests to reduce overhead
- ☐ **Cost Dashboard:** Real-time cost tracking and visualization
- ☐ **Budget Alerts:** Notify when approaching spend limits

B. Performance Optimization

- ☐ **Response Caching:** Cache identical or similar queries
- ☐ **Streaming:** Improve perceived performance with streaming
- ☐ **Lazy Loading:** Load components on-demand
- ☐ **Database Indexing:** Optimize queries with proper indexes
- ☐ **CDN Integration:** Serve static assets from CDN
- ☐ **Compression:** Gzip/Brotli for API responses

C. Scalability Architecture

- ☐ **Stateless Design:** Enable horizontal scaling
- ☐ **Load Balancing:** Distribute traffic across instances
- ☐ **Database Scaling:** Connection pooling, read replicas
- ☐ **Queue System:** Async processing for long-running tasks (Celery, RabbitMQ, Redis)
- ☐ **Auto-scaling:** Scale resources based on demand (if cloud-deployed)

D. Fallback & Reliability

- ☐ **Model Fallback:** Switch to alternative model on failure
 - ☐ **Retry Logic:** Exponential backoff for transient errors
 - ☐ **Circuit Breaker:** Prevent cascading failures
 - ☐ **Graceful Degradation:** Reduced functionality instead of complete failure
 - ☐ **Health Checks:** Monitor service availability
-

10. TESTING & QUALITY ASSURANCE (MUST-HAVE)

A. Unit Testing

- ☐ **Backend Tests:** Test individual functions and modules (pytest, Jest)

- ☐ **Frontend Tests:** Component tests (React Testing Library, Vitest)
- ☐ **Coverage Target:** Minimum 60% code coverage
- ☐ **Mocking:** Mock LLM responses for deterministic tests
- ☐ **Edge Cases:** Test error handling and boundary conditions

B. Integration Testing

- ☐ **API Tests:** Test endpoints with realistic payloads
- ☐ **Database Tests:** Verify CRUD operations
- ☐ **LLM Integration:** Test with real or mocked LLM calls
- ☐ **Vector DB Integration:** Test embedding and retrieval flows

C. End-to-End (E2E) Testing

- ☐ **User Flows:** Test complete user journeys (Playwright, Cypress)
- ☐ **Multi-step Workflows:** Test complex agent interactions
- ☐ **Cross-browser Testing:** Chrome, Firefox, Safari, Edge
- ☐ **Performance Testing:** Load and stress testing (Locust, k6)

D. LLM-Specific Testing

- ☐ **Evaluation Dataset:** 50+ test cases with ground truth answers
- ☐ **Regression Tests:** Ensure updates don't break existing functionality
- ☐ **Hallucination Detection:** Test with misleading or out-of-scope queries
- ☐ **Adversarial Testing:** Try to jailbreak or manipulate the system
- ☐ **Multi-language Testing:** Test with English and Arabic inputs

E. Manual Testing

- ☐ **User Acceptance Testing (UAT):** Real users test the application
- ☐ **Edge Case Discovery:** Manual exploration to find unexpected behaviors
- ☐ **Usability Testing:** Evaluate ease of use and intuitiveness
- ☐ **Security Testing:** Manual penetration testing for vulnerabilities

11. DEPLOYMENT & DEVOPS (MUST-HAVE)

A. Containerization

- ☐ **Docker:** Application containerized with Dockerfile
- ☐ **Docker Compose:** Multi-service orchestration for local dev
- ☐ **Multi-stage Builds:** Optimized image size

- ☐ **Environment Variables:** Externalize configuration

B. Cloud Deployment (Choose ONE platform)

- ☐ **AWS:** EC2, ECS, Lambda, S3, RDS, Bedrock
- ☐ **Azure:** App Service, Container Instances, Cosmos DB, OpenAI Service
- ☐ **Google Cloud:** Cloud Run, GKE, Cloud SQL, Vertex AI
- ☐ **Vercel/Netlify:** Serverless frontend + API routes
- ☐ **Railway/Render:** Simplified full-stack deployment

C. Infrastructure as Code (IaC) (Optional but recommended)

- ☐ **Terraform:** Define infrastructure declaratively
- ☐ **CloudFormation:** AWS-native IaC
- ☐ **Pulumi:** Multi-cloud IaC with programming languages

D. CI/CD Pipeline

- ☐ **Automated Testing:** Run tests on every commit (GitHub Actions, GitLab CI)
- ☐ **Build Automation:** Compile and package application
- ☐ **Automated Deployment:** Deploy to staging/production on merge
- ☐ **Rollback Strategy:** Quick rollback on deployment failure
- ☐ **Branch Strategy:** Main/develop branches with feature branches

E. Secrets Management

- ☐ **Environment Variables:** API keys stored in .env (never in code)
- ☐ **Secrets Manager:** AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault
- ☐ **Rotate Secrets:** Regular API key rotation policy
- ☐ **Least Privilege:** IAM roles with minimal required permissions

F. Monitoring & Logging (Infrastructure)

- ☐ **Application Logs:** Structured logging (JSON) with log levels
 - ☐ **Centralized Logging:** CloudWatch, ELK Stack, or Datadog
 - ☐ **Uptime Monitoring:** Pingdom, UptimeRobot, or built-in
 - ☐ **Error Tracking:** Sentry, Rollbar, or similar
 - ☐ **APM:** Application Performance Monitoring (New Relic, Datadog APM)
-

12. DOMAIN-SPECIFIC IMPLEMENTATIONS (Choose based on project)

A. Conversational AI / Chatbots

- ☐ **Multi-turn Conversations:** Maintain context across interactions
- ☐ **Intent Recognition:** Understand user goals
- ☐ **Entity Extraction:** Identify key information in queries
- ☐ **Slot Filling:** Collect required information through conversation
- ☐ **Fallback Handling:** Graceful responses for out-of-scope queries

B. Document Intelligence

- ☐ **Document Upload:** PDF, DOCX, images
- ☐ **OCR Integration:** Tesseract, AWS Textract, Azure Document Intelligence
- ☐ **Layout Understanding:** Preserve tables, headers, lists
- ☐ **Summarization:** Generate executive summaries
- ☐ **Q&A over Documents:** RAG-based question answering

C. Search & Recommendations

- ☐ **Semantic Search:** Vector-based similarity search
- ☐ **Filters & Facets:** Refine results by categories
- ☐ **Personalization:** Tailor results to user preferences
- ☐ **Related Items:** Recommend similar content
- ☐ **Search Analytics:** Track popular queries and click-through rates

D. Code Generation & Analysis

- ☐ **Code Completion:** Suggest code snippets
- ☐ **Code Explanation:** Describe what code does
- ☐ **Bug Detection:** Identify potential issues
- ☐ **Code Translation:** Convert between languages
- ☐ **Sandbox Execution:** Run generated code safely

E. Content Creation

- ☐ **Text Generation:** Articles, emails, social posts
- ☐ **Style Transfer:** Match brand voice and tone
- ☐ **Translation:** Multi-language content
- ☐ **Content Variations:** A/B test copy
- ☐ **SEO Optimization:** Keywords and meta descriptions

F. Data Analysis & Insights

- ☐ **SQL Generation:** Natural language to SQL queries
 - ☐ **Chart Generation:** Visualize data based on text queries
 - ☐ **Report Generation:** Automated insights from data
 - ☐ **Anomaly Detection:** Identify unusual patterns
 - ☐ **Predictive Analytics:** Forecast trends
-

13. DOCUMENTATION & KNOWLEDGE TRANSFER (MUST-HAVE)

A. User Documentation

- ☐ **User Guide:** How to use the application (screenshots/videos)
- ☐ **FAQ:** Common questions and answers
- ☐ **Troubleshooting:** Solutions to common issues
- ☐ **Tutorial Videos:** Walkthrough of key features (optional)
- ☐ **Onboarding Flow:** In-app guidance for new users

B. Technical Documentation

- ☐ **Architecture Diagram:** System overview and component relationships
- ☐ **API Documentation:** OpenAPI/Swagger spec for all endpoints
- ☐ **Database Schema:** ER diagram and table descriptions
- ☐ **Deployment Guide:** Step-by-step deployment instructions
- ☐ **Configuration Guide:** All environment variables explained

C. Code Documentation

- ☐ **README:** Project overview, setup, and quick start
- ☐ **Inline Comments:** Explain complex logic
- ☐ **Docstrings:** Function/class documentation
- ☐ **Contributing Guide:** How to contribute to the project
- ☐ **Changelog:** Version history and release notes

D. AI-Specific Documentation

- ☐ **Prompt Library:** Document all system prompts used
- ☐ **Model Selection Rationale:** Why specific models chosen
- ☐ **Evaluation Results:** Metrics and benchmarks
- ☐ **Known Limitations:** What the system cannot do

- ☐ **Future Improvements:** Roadmap of planned features

E. Presentation Materials

- ☐ **Project Pitch Deck:** Slide deck explaining project (10-15 slides)
 - ☐ **Demo Video:** 3-5 minute walkthrough
 - ☐ **Business Case:** Problem, solution, market, monetization
 - ☐ **Technical Deep-dive:** For technical audience
-

14. INNOVATION & DIFFERENTIATION (RECOMMENDED)

A. Unique Value Proposition

- ☐ **Clear Differentiation:** What makes this project unique?
- ☐ **Competitive Advantage:** Why is this better than alternatives?
- ☐ **Target Audience:** Who specifically benefits most?
- ☐ **Measurable Impact:** Quantify the value delivered

B. Advanced AI Techniques (Choose minimum 1 to stand out)

- ☐ **Mixture of Agents (MoA):** Combine multiple LLMs for better results
- ☐ **Chain-of-Density Prompting:** Iteratively refine summaries
- ☐ **Graph RAG:** Knowledge graphs for complex relationship queries
- ☐ **Meta-prompting:** LLM generates its own prompts
- ☐ **Constitutional AI:** Self-critique and refinement loops
- ☐ **Retrieval-Interleaved Generation:** Alternate between retrieval and generation
- ☐ **Active Learning:** System identifies uncertain cases for human review

C. Arabic Language Excellence (If targeting Arabic market)

- ☐ **Arabic NLP Models:** AraBERT, AraGPT, or Jais for better Arabic understanding
- ☐ **Dialect Support:** MSA, Egyptian, Gulf, Levantine dialects
- ☐ **Right-to-Left (RTL) UI:** Proper Arabic text rendering
- ☐ **Arabic-specific Prompting:** Prompts optimized for Arabic LLMs
- ☐ **Bilingual Context:** Seamlessly switch between Arabic and English

D. Emerging Technologies (Optional stretch goals)

- ☐ **Model Context Protocol (MCP):** Integrate MCP servers for extended functionality
- ☐ **Reasoning Models:** OpenAI o1, o3-mini for complex problem-solving
- ☐ **Anthropic Claude with Tools:** Advanced function calling and computer use

- ☐ **Local LLM Deployment:** Ollama with Llama 3.2, Qwen, or Mistral
 - ☐ **Edge AI:** Run small models on device (Llama 3.2 1B/3B)
 - ☐ **Workflow Automation:** n8n or Zapier integration for complex workflows
-

SCORING & EVALUATION FRAMEWORK

Tier Classification System

BRONZE TIER - Functional MVP (60-74%)

Minimum requirements to pass:

- All MUST-HAVE sections implemented at basic level
- LLM API integration working
- Basic RAG implementation (simple chunking + vector search)
- Single agent with 3+ tools
- Basic security (input validation, API keys secure)
- Deployed to cloud and accessible
- Basic monitoring and logging
- Comprehensive documentation

SILVER TIER - Production-Ready (75-89%)

Bronze + Advanced implementations:

- Multi-agent system (minimum 3 agents)
- Advanced RAG features (hybrid retrieval + reranking)
- 2+ multimodal capabilities
- Comprehensive security (OWASP LLM Top 10 coverage)
- Full observability platform integration
- Quality evaluation metrics implemented
- Excellent UX with streaming, citations, feedback
- CI/CD pipeline with automated testing
- Cost optimization strategies

GOLD TIER - Industry-Leading (90-100%)

Silver + Innovation & Excellence:

- Sophisticated multi-agent orchestration (5+ agents, hierarchical or swarm)
 - Graph RAG or other advanced RAG architecture
 - 3+ multimodal capabilities seamlessly integrated
 - LLM-as-a-Judge evaluation + Human evaluation loop
 - Advanced agent features (self-reflection, replanning, learning)
 - Arabic language excellence (if applicable)
 - Unique innovation or emerging technology
 - Performance benchmarks exceeding alternatives
 - Clear path to monetization and scaling
 - Publication-ready documentation and materials
-

PROJECT CATEGORIES & SUGGESTED FEATURES

Category A: Conversational AI Agents

Examples: Customer support, HR assistant, sales copilot, legal advisor

- Multi-turn conversation management
- CRM/ticketing system integration
- Sentiment analysis
- Escalation to human agents
- Analytics dashboard

Category B: Knowledge Management & RAG

Examples: Documentation assistant, research tool, internal wiki Q&A

- Multi-source ingestion (Confluence, Notion, Drive, SharePoint)
- Advanced retrieval strategies
- Citation and source tracking

- Knowledge graph visualization
- Collaborative features

Category C: Content Creation & Marketing

Examples: Content copilot, email campaign generator, social media planner

- Brand voice consistency
- SEO optimization
- Multi-platform adaptation
- A/B testing
- Performance analytics

Category D: Developer Tools & Productivity

Examples: Code assistant, documentation generator, API testing copilot

- IDE integration
- Code execution sandbox
- Git integration
- Automated testing generation
- Technical documentation

Category E: Specialized Copilots

Examples: Medical scribe, legal contract review, financial analysis

- Domain-specific knowledge bases
- Compliance and regulatory checks
- Specialized terminology handling
- Professional workflow integration
- Audit trails

Category F: Multi-Agent Marketplaces/Platforms

Examples: Agent marketplace, workflow builder, automation platform

- Agent discovery and hiring

- Visual workflow builder
 - Inter-agent communication protocol
 - Usage analytics and billing
 - Template library
-

ARABIC MARKET CONSIDERATIONS

SAEG Arabic-First Features

If targeting MENA markets, prioritize:

- ☐ **Arabic LLMs:** Jais, AceGPT, or fine-tuned models
 - ☐ **Dialect Support:** Understand colloquial Arabic
 - ☐ **Cultural Context:** Appropriate responses for Arab culture
 - ☐ **RTL UI:** Proper right-to-left interface
 - ☐ **Arabic SEO:** Optimize for Arabic search
 - ☐ **Local Payment:** Fawry, PayMob, or regional gateways
 - ☐ **Arabic Voice:** TTS/STT for Arabic dialects
 - ☐ **Islamic Finance:** Shariah-compliant features (if fintech)
-

TECHNOLOGY STACK RECOMMENDATIONS

Backend Options

- **Python:** FastAPI, Django, Flask
- **.NET:** ASP.NET Core, Blazor
- **Node.js:** Express, NestJS, Next.js API routes
- **PHP:** Laravel, Symfony

Frontend Options

- **React:** Next.js, Vite, Create React App
- **Vue:** Nuxt, Vite
- **Angular:** Angular CLI
- **Blazor:** .NET web UI

AI/ML Libraries

- **Python:** LangChain, LlamaIndex, Haystack, AutoGen, CrewAI
- **.NET:** Semantic Kernel, ML.NET
- **JavaScript:** LangChain.js, LlamaIndex.js, Vercel AI SDK

Vector Databases

- **Cloud:** Pinecone, Weaviate, Qdrant Cloud
- **Self-hosted:** Chroma, FAISS, Milvus
- **Postgres Extension:** pgvector

Observability

- **LLM-specific:** Langfuse, LangSmith, Helicone, Phoenix
 - **General:** Datadog, New Relic, Grafana + Prometheus
 - **Open-source:** OpenTelemetry + Jaeger
-

WEEKLY MILESTONE CHECKLIST

Week 1: Foundation & Planning

- ☐ Project proposal and architecture finalized
- ☐ Technology stack selected
- ☐ Repository setup with CI/CD skeleton
- ☐ Basic authentication and database schema
- ☐ First LLM API call working
- ☐ Team roles and responsibilities defined

Week 2: Core AI Implementation

- ☐ RAG pipeline fully functional
- ☐ Single agent with 3+ tools working
- ☐ Basic UI for interaction
- ☐ Vector database integrated
- ☐ Prompt engineering and initial testing

- ☐ Error handling and logging in place

Week 3: Advanced Features & Multi-Agent

- ☐ Multi-agent system implemented
- ☐ Multimodal capabilities added
- ☐ Security features (guardrails, input validation)
- ☐ Observability platform integrated
- ☐ Performance optimization
- ☐ Testing suite (unit + integration)

Week 4: Polish, Testing & Deployment

- ☐ E2E testing and bug fixes
 - ☐ Documentation completed
 - ☐ Production deployment
 - ☐ Evaluation metrics calculated
 - ☐ Demo video recorded
 - ☐ Presentation deck prepared
-

PROHIBITED/DISCOURAGED PRACTICES

✗ Do NOT:

- ☐ Use localStorage for sensitive data (API keys, user tokens)
- ☐ Hardcode API keys in source code
- ☐ Deploy without rate limiting
- ☐ Skip input validation and sanitization
- ☐ Use only keyword search (no semantic search)
- ☐ Ignore error handling and edge cases
- ☐ Skip testing "because it's a demo" (it's NOT a demo!)
- ☐ Copy-paste code without understanding
- ☐ Overcomplicate: focus on working features over buzzwords
- ☐ Neglect documentation until the last day

⚠ Common Pitfalls to Avoid:

- ☐ **Scope creep:** Start small, iterate
- ☐ **Over-engineering:** Build what you need, not what you might need
- ☐ **Ignoring costs:** Monitor token usage from day 1

- ☐ **Poor team coordination:** Daily standups and clear ownership
 - ☐ **Late integration:** Integrate continuously, don't wait for the end
 - ☐ **Ignoring user feedback:** Test with real users early
 - ☐ **Underestimating documentation:** Budget time for docs from start
-

SUCCESS METRICS & KPIs

Technical Metrics

- ☐ **Latency:** P95 response time < 3 seconds
- ☐ **Availability:** 99%+ uptime during evaluation period
- ☐ **Error Rate:** < 5% of requests fail
- ☐ **Test Coverage:** > 60% code coverage
- ☐ **Security Score:** 0 critical vulnerabilities

AI Quality Metrics

- ☐ **Faithfulness:** > 85% answers grounded in context
- ☐ **Answer Relevancy:** > 80% responses directly address query
- ☐ **User Satisfaction:** > 70% positive feedback (thumbs up)
- ☐ **Task Success:** > 75% agent tasks completed successfully

User Experience Metrics

- ☐ **Time to First Response:** < 2 seconds
- ☐ **User Retention:** Users return for multiple sessions
- ☐ **Feature Adoption:** Core features used in 80%+ of sessions
- ☐ **Documentation Clarity:** Users can self-onboard without help

Business Metrics

- ☐ **Cost per Query:** < \$0.10 (optimize for profitability)
 - ☐ **Scalability:** Handles 100+ concurrent users
 - ☐ **Market Validation:** Clear value proposition and target users
 - ☐ **Monetization Path:** Pricing model defined
-

BONUS POINTS & DIFFERENTIATORS

- 🌟 **Extra Credit Features (Not required but impressive)**

- ☐ **Mobile App:** Native iOS/Android or React Native
- ☐ **Browser Extension:** Chrome/Firefox extension
- ☐ **API for Third-party:** Well-documented public API
- ☐ **Webhook Integration:** Slack, Discord, Teams notifications
- ☐ **Plugin Architecture:** Extensible with custom agents/tools
- ☐ **White-label:** Multi-tenant with custom branding
- ☐ **Analytics Dashboard:** Admin panel with usage insights
- ☐ **Community Features:** User forums, feedback boards
- ☐ **Gamification:** Badges, leaderboards for engagement
- ☐ **Offline Mode:** Progressive Web App with offline capabilities

Innovation Categories

- **Most Creative Use of AI:** Novel application of LLMs
 - **Best Arabic Implementation:** Excellence in Arabic NLP
 - **Best Multi-Agent System:** Sophisticated agent orchestration
 - **Best User Experience:** Intuitive, delightful interface
 - **Most Production-Ready:** Closest to market launch
 - **Best Security Implementation:** Comprehensive OWASP coverage
 - **Best Evaluation Framework:** Rigorous quality measurement
 - **Social Impact:** Addresses meaningful societal problem
-

RESOURCES & SUPPORT

Provided by Institution

-  OpenAI API credits
-  Azure/AWS cloud credits
-  GitHub Copilot licenses
-  Vector database subscriptions (Pinecone, Weaviate, etc.)
-  Observability platform access (Langfuse, LangSmith)
-  Course materials and recorded sessions

Recommended Tools (Free Tier Available)

- GitHub (version control)
- Vercel/Netlify (frontend hosting)
- Railway/Render (backend hosting)
- Supabase/PlanetScale (database)
- Postman (API testing)
- Figma (design)
- Canva (presentations)
- Loom (demo videos)

Getting Help

- Daily standup with instructors
 - Office hours for technical questions
 - Peer code reviews
 - Industry mentor sessions
 - Slack/Teams support channel
-

FINAL CHECKLIST BEFORE SUBMISSION

☒ Pre-Submission Validation

- ☐ All MUST-HAVE sections completed
- ☐ Application deployed and publicly accessible
- ☐ Demo video recorded (3-5 minutes)
- ☐ All documentation in repository
- ☐ README has setup instructions
- ☐ No API keys in code or commits
- ☐ Tests passing in CI/CD
- ☐ No critical security vulnerabilities
- ☐ Presentation deck ready
- ☐ Team practiced demo run-through

Deliverables Checklist

- ☐ **Source Code:** GitHub repository with clean commits
 - ☐ **Live Demo:** Deployed application URL
 - ☐ **Documentation:** README, API docs, architecture diagram
 - ☐ **Demo Video:** YouTube/Vimeo unlisted link
 - ☐ **Presentation:** Slide deck (PDF)
 - ☐ **Evaluation Report:** Metrics and test results
 - ☐ **User Guide:** How to use the application
 - ☐ **Known Issues:** Limitations and future work
-

CLOSING NOTES

Philosophy

This checklist is comprehensive but flexible. The goal is to build a **production-ready MVP**, not a demo. Focus on:

1. **Solving a real problem** for real users
2. **Quality over quantity:** Better to have 5 excellent features than 15 mediocre ones
3. **Security and safety first:** Protect users and data
4. **Measure everything:** You can't improve what you don't measure
5. **Document relentlessly:** Future you (and others) will thank you

Scoring Guidance

- **Bronze (60-74%):** Meets all fundamental requirements, working application
- **Silver (75-89%):** Production-ready with advanced features and excellent execution
- **Gold (90-100%):** Industry-leading quality with innovation and potential for market success

Remember

- Start with a clear problem and user need
- Build iteratively: Week 1 = Foundation, Week 2 = Core, Week 3 = Advanced, Week 4 = Polish
- Test early and often
- Prioritize ruthlessly: What delivers most value?

- Collaborate effectively: Clear roles, daily communication
- Document as you go, not at the end
- Security and evaluation are not optional
- This is your portfolio piece: Make it count!

Good luck building the future of AI applications! 🚀

Checklist Version 1.0 | January 2026 | Based on ITI Gen-AI Training Curriculum Aligned with: OWASP LLM Top 10 2025, Current Industry Standards, Freelancing Market Trends